

Chapitre 7

Phase de clôture

Introduction

Ce chapitre est consacré à la présentation des technologies utilisées pour la réalisation pratique du projet **NeoLeadge**. Dans un premier temps, nous détaillerons les choix technologiques ainsi que l’environnement matériel et logiciel mis en place. Ensuite, nous présenterons les langages et frameworks retenus. Enfin, nous illustrerons l’architecture logicielle et physique de l’application, ainsi que la procédure de déploiement.

7.1 Environnement de développement

Dans cette section, nous présentons l’environnement matériel et logiciel utilisé pour la mise en place de notre projet.

7.1.1 L’environnement matériel

TABLE 7.1 – Caractéristiques de l’ordinateur utilisé

<i>Caractéristique</i>	Ordinateur 1
Marque	HP
Propriétaire	—
Processeur	Intel Core i7
RAM	24,00 Go
Disque dur	512 Go SSD
Système d’exploitation	Windows 11

7.1.2 L'environnement logiciel

Cette section a pour objectif de présenter les différents outils utilisés dans le développement de notre projet.

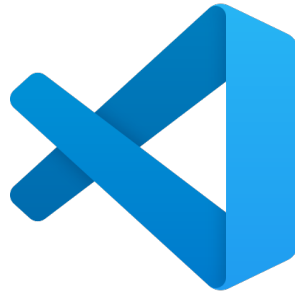


FIGURE 7.1 – Logo de Visual Studio Code

1) IDE de développement : Visual Studio Code est un éditeur de code extensible développé par Microsoft, compatible avec Windows, Linux et macOS. Il propose la prise en charge du débogage, la coloration syntaxique, l'autocomplétion, l'intégration Git et un large écosystème d'extensions (ESLint, Prettier, Volar pour Vue, Prisma).

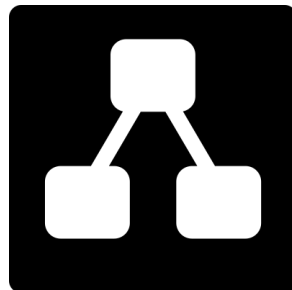


FIGURE 7.2 – Logo de draw.io (diagrams.net)

2) Outil de conception : draw.io (diagrams.net) est un outil de diagramme open source permettant de réaliser les diagrammes UML, les schémas d'architecture et les modèles de base de données. Il a servi à concevoir les diagrammes de cas d'utilisation, de classes et l'architecture de l'application.



FIGURE 7.3 – Logo de Prisma Studio

3) Administration de la base de données : Prisma Studio est une interface graphique fournie avec l'ORM Prisma. Elle permet de visualiser et d'éditer les données de la base PostgreSQL directement depuis le navigateur, ainsi que d'inspecter les relations entre les 44 modèles du schéma.

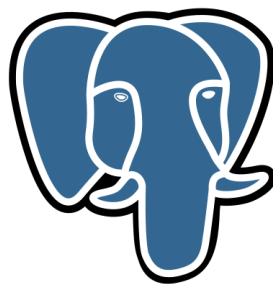


FIGURE 7.4 – Logo de PostgreSQL

4) Système de gestion de base de données : PostgreSQL est un système de gestion de bases de données relationnelles (SGBDR) open source, réputé pour sa robustesse, sa conformité au standard SQL et son extensibilité. NeoLeadge utilise PostgreSQL 16 comme base de données principale.



FIGURE 7.5 – Logo de pgvector

pgvector est une extension de PostgreSQL qui ajoute un type de données **vector** et des index de similarité (HNSW). Elle est utilisée par NeoLeadge pour la recherche sémantique : les transcriptions de réunions et les réponses du questionnaire sont indexées sous forme de vecteurs (embeddings).

multilingual-e5-small, 384 dimensions) afin que l'IA retrouve les passages les plus pertinents lors de la génération du cahier des charges.

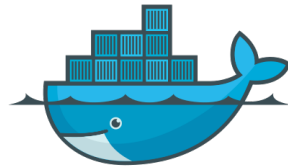


FIGURE 7.6 – Logo de Docker

5) Conteneurisation et déploiement local : Docker est une plateforme de conteneurisation qui permet d'empaqueter une application et ses dépendances dans des conteneurs isolés. Avec Docker Compose, l'ensemble de la stack NeoLedger (serveur NestJS, frontend, base PostgreSQL, service de transcription Python, reverse proxy Caddy) est orchestré et démarré de façon reproductible aussi bien en local qu'en production.



FIGURE 7.7 – Logo d'Overleaf

6) Outil de réalisation du rapport : Overleaf est une plateforme web gratuite qui permet de créer et de modifier des documents $\text{\LaTeX}$ directement en ligne, sans installation, avec des fonctionnalités de travail collaboratif sur un même document.



FIGURE 7.8 – Logo de L_AT_EX

L_AT_EX est un langage et un système de mise en forme de documents, constitué d'un ensemble de macro-commandes conçues pour simplifier la production de documents structurés de qualité professionnelle.

7.2 Les langages de programmation et frameworks

7.2.1 Langages de programmation



FIGURE 7.9 – Logo de TypeScript

TypeScript est un sur-ensemble typé de JavaScript développé par Microsoft. Il ajoute un système de types statiques qui sécurise le code et facilite la maintenance. NeoLedger est écrit intégralement en TypeScript, côté serveur (NestJS) comme côté client (Vue 3).



FIGURE 7.10 – Logo de HTML5

HTML est un langage de balisage utilisé pour créer et structurer les pages web. Il permet d'écrire de l'hypertexte tout en définissant la structure sémantique du contenu d'une page.



FIGURE 7.11 – Logo de CSS3

Le CSS est un langage utilisé pour styliser les pages HTML. Il sépare le contenu de la mise en forme. Dans NeoLeadge, le style s'appuie sur Tailwind CSS et sur les composants de la bibliothèque NeoLibrary.



FIGURE 7.12 – Logo de Python

Python est un langage de haut niveau réputé pour sa lisibilité et son riche écosystème scientifique. Il est utilisé pour le microservice de transcription NeoLeadge (FastAPI + faster-whisper + SpeechBrain) et pour le service d'embeddings sémantiques.

7.2.2 Les frameworks et bibliothèques



FIGURE 7.13 – Logo de Node.js

Node.js est un environnement d'exécution JavaScript côté serveur, basé sur le moteur V8 de Chrome. Son fonctionnement asynchrone et son écosystème npm en font une base efficace pour les API REST. Le serveur NeoLeadge s'exécute sur Node.js.



FIGURE 7.14 – Logo de NestJS

NestJS est un framework Node.js progressif pour construire des applications serveur efficaces et maintenables. Il s'appuie sur TypeScript et une architecture modulaire (modules, contrôleurs, services, injection de dépendances). NeoLeadge utilise NestJS 11 pour son API REST, ses WebSockets (Socket.IO) et ses tâches planifiées.



FIGURE 7.15 – Logo de Vue.js

Vue.js est un framework JavaScript progressif open source utilisé pour construire des interfaces utilisateur et des applications monopages (SPA). La version 3 introduit la Composition API et un système de réactivité performant. NeoLeadge utilise Vue 3 pour l'ensemble de son interface.



FIGURE 7.16 – Logo de Vite

Vite est un outil de build et un serveur de développement nouvelle géné-

ration. Il offre un démarrage quasi instantané et un rechargement à chaud (HMR) rapide grâce aux modules ES natifs. Il sert de bundler au frontend NeoLeadge.



FIGURE 7.17 – Logo de Pinia

Pinia est la bibliothèque officielle de gestion d'état pour Vue 3. Elle centralise l'état partagé de l'application (utilisateur, projets, notifications) de manière typée et réactive.



FIGURE 7.18 – Logo de PrimeVue / NeoLibrary

NeoLibrary est la bibliothèque de composants interne du projet, construite au-dessus de PrimeVue 4 (boutons, tableaux, formulaires, fenêtres modales, calendriers) avec un style Tailwind personnalisé. Elle garantit une interface cohérente sur l'ensemble de l'application.



FIGURE 7.19 – Logo de Prisma

Prisma est un ORM (mapping objet-relationnel) moderne pour TypeScript. Il génère un client typé à partir d'un schéma déclaratif, gère les migrations versionnées et sécurise les requêtes. NeoLeadge utilise Prisma 7 avec l'adaptateur `@prisma/adaptre-pg` pour PostgreSQL.



FIGURE 7.20 – Logo de FastAPI

FastAPI est un framework web Python rapide et moderne pour construire des API. NeoLeadge l'utilise pour son microservice de transcription audio (faster-whisper + SpeechBrain) et son point d'accès d'embeddings sémantiques.

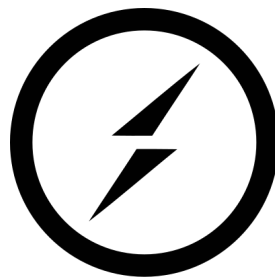


FIGURE 7.21 – Logo de Socket.IO

Socket.IO permet une communication bidirectionnelle en temps réel entre le serveur et les clients. NeoLeadge l'utilise pour les notifications instantanées et la présence collaborative pendant l'édition des questionnaires.

7.2.3 Autres logiciels et services



FIGURE 7.22 – Logo de Z.AI

Z.AI fournit le modèle de langage `glm-4.5-air` (compatible avec l'API OpenAI) utilisé par NeoLeadge pour l'ensemble des fonctionnalités d'IA : analyse des réunions, génération du cahier des charges, proposition de backlog et suggestions d'affectation des tâches.



FIGURE 7.23 – Logo de Postman

Postman est une application qui permet de tester et d’explorer les API de manière simple et efficace, et de partager les tests entre développeurs.



FIGURE 7.24 – Logo de Caddy

Caddy est un serveur web et reverse proxy qui gère automatiquement les certificats TLS via Let’s Encrypt. En production, il route le trafic HTTPS vers le frontend et l’API NeoLedge sur <https://neoleadge.pythagore-init.com>.

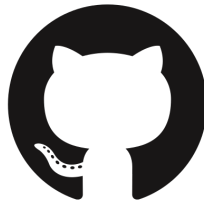


FIGURE 7.25 – Logo de GitHub

GitHub est une plateforme en ligne qui permet d’héberger, de gérer et de collaborer sur des projets utilisant le système de gestion de versions Git. Le dépôt NeoLedge y est hébergé, avec une intégration continue (GitHub Actions) qui exécute les tests à chaque *pull request*.

7.3 Architecture de projet

7.3.1 L'architecture logicielle utilisée

NeoLeadge repose sur une **architecture en couches modulaire** côté serveur (NestJS) et sur une architecture de composants côté client (Vue 3). Côté serveur, chaque domaine fonctionnel est isolé dans un module dédié, structuré en trois responsabilités distinctes :

- **Contrôleur (Controller)**

Rôle : il reçoit les requêtes HTTP, valide les données entrantes (DTO + `ValidationPipe`) et délègue le traitement au service approprié, puis transforme le résultat en réponse HTTP.

- **Service (Service)**

Rôle : il porte la logique métier. Chaque méthode retourne un objet `Result.ok` ou `Result.fail` plutôt que de lever une exception, ce qui rend la gestion d'erreurs explicite et prévisible.

- **Accès aux données (Prisma)**

Rôle : la couche d'accès aux données s'appuie sur l'ORM Prisma, qui encapsule les requêtes vers PostgreSQL et garantit un typage de bout en bout.

Côté client, Vue 3 organise l'interface en composants réutilisables, l'état partagé étant géré par Pinia. La séparation des responsabilités y reste claire : vues (pages), composants, magasins (stores) et services d'accès à l'API.

Architecture logicielle en couches (NestJS)

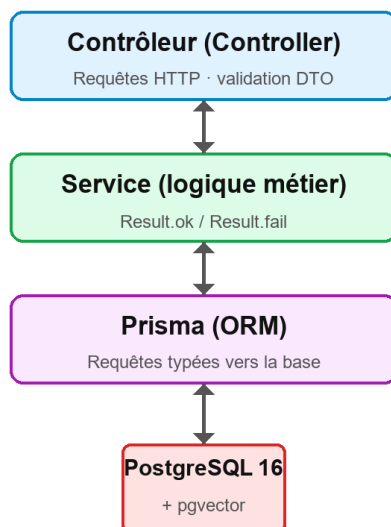


FIGURE 7.26 – Architecture logicielle en couches (NestJS)

7.3.2 Architecture physique

Cette partie présente l'architecture physique de l'application et les technologies déployées. L'application est construite autour de plusieurs services conteneurisés orchestrés par Docker Compose.

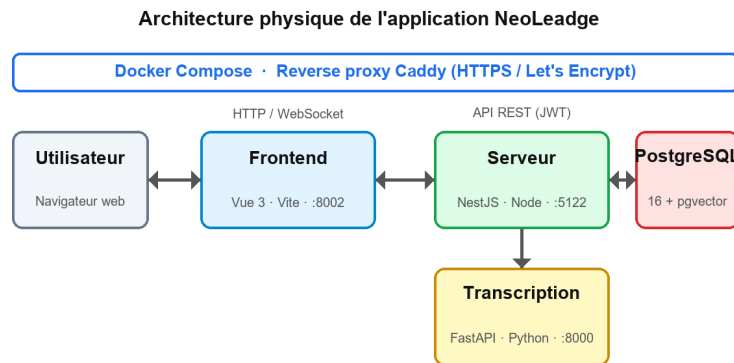


FIGURE 7.27 – Architecture physique de l'application NeoLeadge

L'architecture physique se compose des éléments suivants :

Côté client (Vue 3 + Vite) :

- Application monopage (SPA) servie en production par un conteneur web (port 8002) derrière le reverse proxy Caddy.
- Communication avec l'API via des requêtes HTTP (axios) et des WebSockets (Socket.IO) pour le temps réel.

Côté serveur (NestJS sur Node.js, port 5122) :

- Gestion et traitement des requêtes HTTP entrantes (API REST sécurisée par JWT).
- Connexion et manipulation des données via Prisma vers PostgreSQL.
- Gestion des opérations asynchrones et des tâches planifiées (analyse IA des réunions, échéances, rétention).

Base de données (PostgreSQL 16 + pgvector) :

- Stockage relationnel des données métier (44 modèles).
- Recherche sémantique vectorielle (index HNSW) pour les fonctionnalités d'IA.

Service de transcription (Python / FastAPI, port 8000) :

- Transcription audio des réunions (faster-whisper) et diarisation des locuteurs (SpeechBrain).

- Génération des embeddings sémantiques (`multilingual-e5-small`).

Orchestration et déploiement :

- L'ensemble des services est défini dans un fichier Docker Compose et déployé sur un serveur Linux.
- Le reverse proxy Caddy assure la terminaison TLS (Let's Encrypt) et le routage HTTPS vers le frontend et l'API.